FlairSim

Simulateur de drone sur imagerie aérienne FLAIR-HUB

Documentation technique complète

Architecture, API, diagrammes de séquence

Fil rouge — Mastère Spécialisé Big Data / Intelligence Artificielle
Télécom Paris

20 mars 2026

Basé sur FLAIR-HUB [1] et FlySearch [2]

Table des matières

1 Introduction **3**

2 Architecture haut niveau **3**
2.1 Vue d'ensemble du système . . . 3
2.2 Cycle de vie d'une session . . . 4
2.3 Timeout d'inactivité . . . 4

3 Architecture des modules (bas niveau) **4**
3.1 Hiérarchie des packages . . . 4
3.2 Module `map/` — Gestion géospatiale . . . 4
3.3 Module `drone/` — État du drone et capteurs . . . 5
3.3.1 Modèle de caméra . . . 5
3.4 Module `core/` — Moteur de simulation . . . 5
3.5 Module `server/` — API HTTP REST . . . 6
3.6 Module `web/` — Orchestrateur web . . . 6
3.7 Module `viewer/` — Visualisation Pygame . . . 6
3.8 Module `data/` — Téléchargement automatique . . . 6

4 Architecture du front-end SPA **7**

5 API du serveur simulateur **7**
5.1 `POST /reset` — Démarrer un épisode . . . 8
5.2 `POST /step` — Envoyer une action . . . 8
5.3 `GET /state` — État courant du drone . . . 8
5.4 `GET /snapshot` — Dernière observation cachée . . . 8
5.5 `GET /telemetry` — Journal de vol . . . 8
5.6 `GET /config` — Configuration du simulateur . . . 9
5.7 `GET /scenarios` — Liste des scénarios . . . 9
5.8 `GET /overview` — Image vue d'ensemble . . . 9
5.9 `GET /events` — Flux SSE . . . 9
5.10 Format `ObservationResponse` . . . 9

6 API de l'orchestrateur web **10**
6.1 `GET /` — Redirection vers la SPA . . . 10
6.2 `GET /api/status` — Health check . . . 10
6.3 `GET /api/scenarios` — Liste des scénarios . . . 10
6.4 `GET /api/scenarios/{id}` — Détail d'un scénario . . . 10
6.5 `GET /api/scenarios/{id}/overview` — Image overview . . . 10
6.6 `POST /api/sessions` — Créer une session . . . 11
6.7 `GET /api/sessions` — Lister les sessions . . . 11
6.8 `GET /api/sessions/{id}` — État d'une session . . . 11
6.9 `DELETE /api/sessions/{id}` — Détruire une session . . . 11
6.10 `ANY /api/sessions/{id}/sim/{path}` — Proxy . . . 11
6.11 `GET /api/leaderboard` — Classement . . . 11
6.12 `POST /api/leaderboard` — Soumettre un run . . . 11
6.13 `POST /api/leaderboard/submit` — Alias . . . 12
6.14 `GET /api/leaderboard/{id}` — Détail d'un run . . . 12

7 Leaderboard — Schéma SQLite **12**

8 Format des scénarios YAML **12**

9 Diagrammes de séquence **13**
9.1 Création d'une session humaine (navigateur) . . . 13
9.2 Mouvement fluide (smooth step) via SSE . . . 14
9.3 Session AI depuis un notebook . . . 15
9.4 Spectateur rejoignant une session en cours . . . 16
9.5 Flux de données complet d'un pas de simulation . . . 17

10 Authentification API **17**

11 Données FLAIR-HUB **18**
11.1 Modalités supportées . . . 18
11.2 Téléchargement automatique . . . 18

12 Grille overlay pour VLMs **18**

13 Installation et utilisation **18**
13.1 Installation . . . 18
13.2 Lancer le simulateur (mode standalone) . . . 18
13.3 Lancer la plateforme web . . . 19
13.4 Viewer Pygame . . . 19
13.5 Contrôles clavier . . . 19

14 Tests **19**

15 Structure du projet **20**

16 Références **21**

1 Introduction

FlairSim est un simulateur de drone léger, écrit entièrement en Python, conçu pour survoler de l'imagerie aérienne réelle issue du jeu de données **FLAIR-HUB** [1] publié par l'IGN.

Le système expose deux niveaux d'interface :

1. Un **serveur simulateur** (`flairsim-server`) avec API HTTP REST + SSE pour le pilotage direct par un agent.
2. Un **orchestrateur web** (`flairsim-web`) qui gère des sessions isolées, sert une SPA navigateur, et persiste les résultats dans un leaderboard SQLite.

2 Architecture haut niveau

2.1 Vue d'ensemble du système

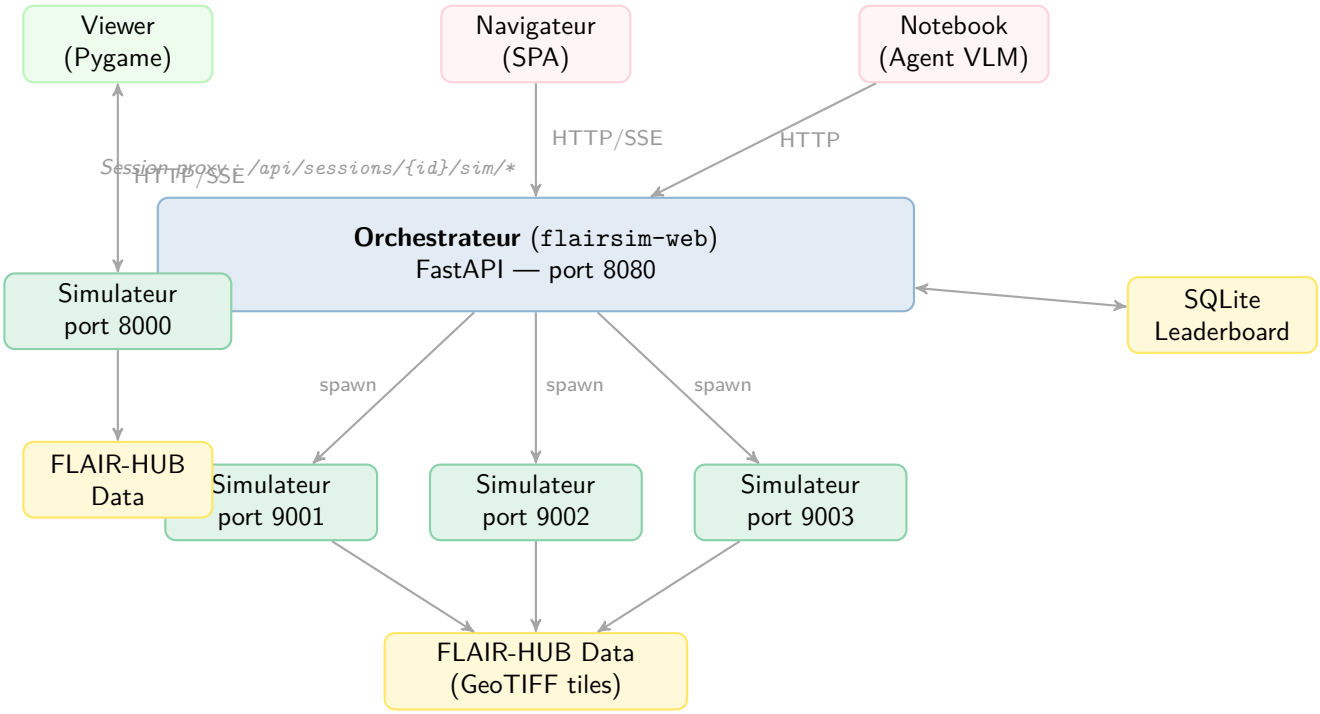

Figure 1 – Architecture deux tiers — Orchestrateur + Simulateurs isolés

Principes de conception :

- **Isolation par sous-processus** : chaque session reçoit son propre processus `python -m flairsim.server`. Aucune interférence entre sessions.
- **Pool de ports** : ports 9001–9099 alloués dynamiquement, recyclés à la destruction. Max 99 sessions concurrentes.
- **Proxy transparent** : l'orchestrateur relaie toutes les requêtes vers le bon sous-processus via `httpx`.
- **SSE proxy** : le flux SSE est relayé en streaming via `httpx.AsyncClient.stream()`.
- **Aucun code VLM sur le serveur** : les agents tournent à l'extérieur (notebooks, scripts). Le serveur fournit uniquement `/reset`, `/step`, `/events`.

2.2 Cycle de vie d'une session

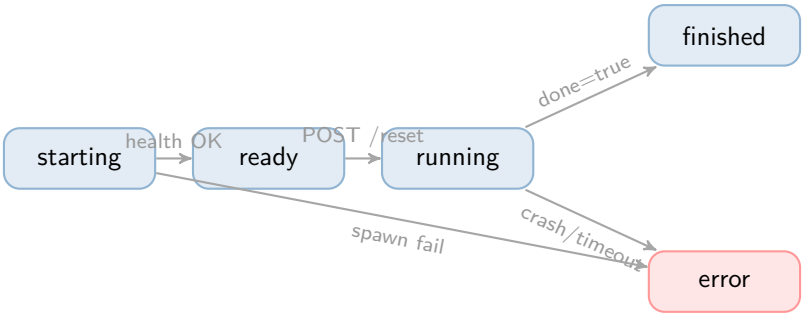

Figure 2 – États d'une session

2.3 Timeout d'inactivité

Chaque requête proxy appelle `touch_session()` qui met à jour `last_activity`. Un task asyncio vérifie toutes les 30 s si une session est inactive depuis plus de 3 minutes (`_IDLE_TIMEOUT = 180`). Les sessions inactives sont détruites automatiquement (processus tué, port libéré).

3 Architecture des modules (bas niveau)

3.1 Hiérarchie des packages

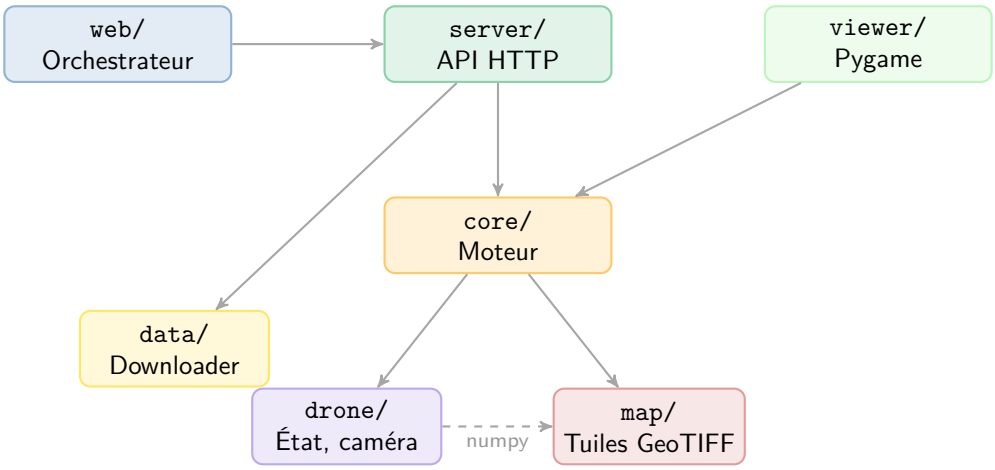

Figure 3 – Dépendances entre packages — flux descendant strict, aucune dépendance circulaire

3.2 Module `map/` — Gestion géospatiale

Classe	Fichier	Rôle
`Modality`	`modality.py`	Enum des 10 modalités FLAIR-HUB avec `ModalitySpec`
`MapBounds`	`map_manager.py`	Rectangle englobant avec intersections
`MapManager`	`map_manager.py`	Moteur spatial : découverte de tuiles, grille, extraction de régions
`TileInfo`	`tile_loader.py`	Métadonnées d'une tuile (path, bounds)
`TileData`	`tile_loader.py`	Tuile chargée : data array + bounds

Table 1 – Classes du module `map/`

Fonctions clés :

— `discover_modalities(parent, domain)` : scanne les répertoires frères pour trouver toutes les modalités disponibles.
— `MapManager.get_region(x, y, half, size)` : extrait et assemble les tuiles en une image continue redimensionnée via PIL LANCZOS.
— Système de coordonnées : EPSG :2154 (Lambert-93, mètres).

3.3 Module `drone/` — État du drone et capteurs

Classe	Fichier	Rôle
`DroneConfig`	`drone.py`	Limites physiques : $z_{\min}$, $z_{\max}$, distance max
`DroneState`	`drone.py`	Position courante (x, y, z), heading, compteurs
`Drone`	`drone.py`	reset, move (avec clampage aux bornes)
`CameraConfig`	`camera.py`	FOV (90°), taille image (500 px)
`CameraModel`	`camera.py`	Géométrie nadir, capture d'image
`TelemetryRecord`	`telemetry.py`	Snapshot gelé par pas de temps
`FlightLog`	`telemetry.py`	Journal de vol complet

Table 2 – Classes du module `drone/`

3.3.1 Modèle de caméra

La caméra pointe toujours au nadir. La couverture au sol dépend de l'altitude :

$$\text{empreinte} = 2 \times z \times \tan\left(\frac{\text{FOV}}{2}\right) \qquad \text{GSD} = \frac{\text{empreinte}}{\text{image_size}} \tag{1}$$

Altitude (m)	Empreinte (m)	GSD (m/px)
10	20	0.04
50	100	0.20
100	200	0.40
200	400	0.80
500	1000	2.00

Table 3 – Couverture au sol (FOV = 90°, image_size = 500 px)

3.4 Module `core/` — Moteur de simulation

Classe	Fichier	Rôle
`ActionType`	`action.py`	Enum : MOVE, FOUND, STOP
`Action`	`action.py`	Commande gelée : $(dx, dy, dz, type)$
`Observation`	`observation.py`	Image + état + done + result
`EpisodeResult`	`observation.py`	success, reason, steps, distance
`FlairSimulator`	`simulator.py`	Classe principale du simulateur
`SimulatorConfig`	`simulator.py`	Configuration globale
`Scenario`	`scenario.py`	Mission YAML complète
`ScenarioLoader`	`scenario.py`	Chargement et cache des scénarios
`GridOverlay`	`grid.py`	Grille $N \times N$ pour prompting VLM

Table 4 – Classes du module `core/`

Conditions de fin d'épisode :

1. **FOUND** : l'agent déclare la cible trouvée. `success` dépend de la distance au target (si scénario actif).
2. **STOP** : arrêt volontaire (`success=False`).
3. **Limite de pas** : `max_steps` atteint (`success=False`).

3.5 Module `server/` — API HTTP REST

Serveur FastAPI créé par la factory `create_app()`. Un seul processus sert une seule simulation. Voir section 5 pour la référence complète des endpoints.

3.6 Module `web/` — Orchestrateur web

Fichier	Classe	Rôle
`app.py`	—	FastAPI app : routes, proxy, middleware
`sessions.py`	`SessionManager`	Spawn/destroy subprocess, pool de ports, health check, idle timeout
`leaderboard.py`	`Leaderboard`	SQLite CRUD : submit, query, ranking
`cli.py`	—	Parsing CLI
`static/`	—	SPA vanilla JS (HTML + CSS + JS)

Table 5 – Fichiers du module `web/`

3.7 Module `viewer/` — Visualisation Pygame

Trois modes de fonctionnement :

Mode	Flag	Description
local	`-mode local`	Simulateur local, vol direct au clavier
observe	`-mode observe`	Connexion SSE au serveur, lecture seule
fly	`-mode fly`	Connexion HTTP, envoi de `POST /step`

Table 6 – Modes du viewer

3.8 Module `data/` — Téléchargement automatique

`HubDownloader` télécharge les données depuis HuggingFace (`IGNF/FLAIR-HUB`). Workflow : `hf_hub_download()` → `zipfile.extractall()` → suppression du ZIP. Nettoyage automatique (`shutil.rmtree`) à l'arrêt du serveur.

4 Architecture du front-end SPA

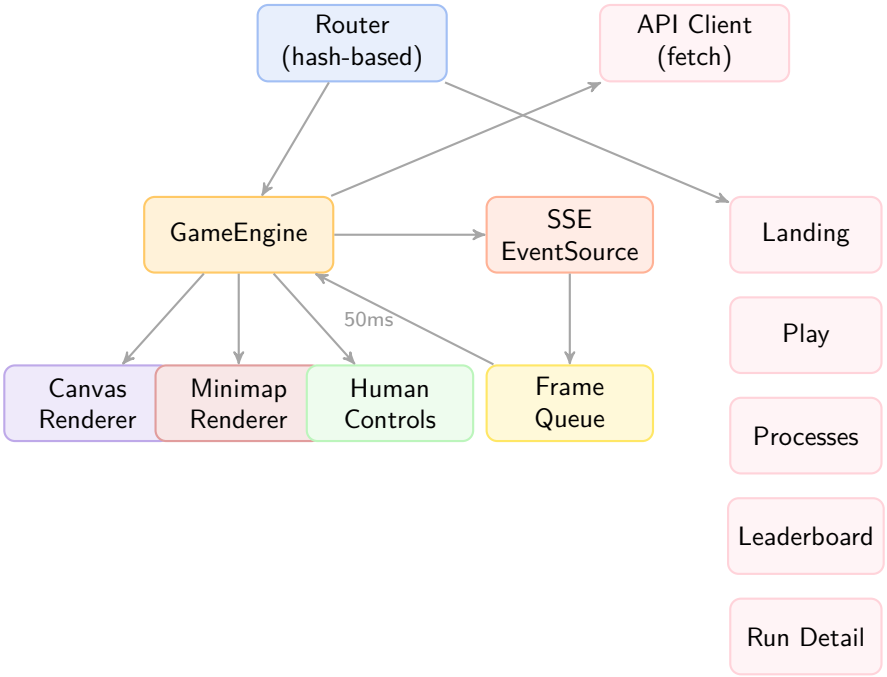

Figure 4 – Architecture du front-end SPA (vanilla JS, 1648 lignes)

Classes principales :

- `GameEngine` : gère le cycle de jeu, la connexion SSE, le rendu frame-by-frame, la détection de fin d'épisode.
- `CanvasRenderer` : dessine l'image cockpit (700×700) avec un réticule central.
- `MinimapRenderer` : minimap 180×180 avec trajectoire, drone, cible.
- `HumanControls` : AZERTY/QWERTY, slider de pas, debounce.

Mécanisme de rendu fluide :

1. Le serveur décompose un mouvement en micro-pas (∼10 m).
2. Chaque micro-pas est diffusé via SSE avec 50 ms de délai.
3. Le JS empile les frames dans `_frameQueue[]`.
4. Un timer à 50 ms dépile une frame à la fois (∼20 FPS).
5. L'input utilisateur est bloqué (`_processing=true`) jusqu'à la réception du pas logique final.

5 API du serveur simulateur

Tous les endpoints sont servis par un unique processus simulateur. L'URL de base est `http://127.0.0.1:8000` en mode standalone, ou `http://127.0.0.1:{port}` en mode sous-processus.

5.1 `POST /reset` — Démarrer un épisode

Champ	Détail
Query params	`?grid=N` (int, optionnel) — taille de la grille overlay
Request body	`ResetRequest` (optionnel) `x` : float|null — position de départ est `y` : float|null — position de départ nord `z` : float|null — altitude de départ `scenario_id` : string|null — scénario à charger
Response	`ObservationResponse` (voir section 5.10)
Erreurs	400 (scénarios non configurés), 404 (scénario introuvable)
Effets	Broadcast SSE à tous les abonnés

5.2 `POST /step` — Envoyer une action

Champ	Détail
Query params	`?grid=N` (int, optionnel)

Request body	`StepRequest`
	`dx` : float (défaut 0.0) — déplacement est
	`dy` : float (défaut 0.0) — déplacement nord
	`dz` : float (défaut 0.0) — déplacement altitude
	`action_type` : `"move"`|`"found"`|`"stop"`
	`reason` : string|null — raisonnement de l'agent

Response	`ObservationResponse`
Erreurs	409 (pas d'épisode actif), 422 (type d'action invalide)
Smooth step	Si `smooth_step_size > 0` et distance > seuil : décomposition en N micro-pas, broadcast SSE intermédiaire avec `asyncio.sleep(0.05)`. Compteur de pas : +1 seul.

5.3 `GET /state` — État courant du drone

Response	`DroneStateResponse : {x, y, z, heading, step_count, total_distance}`
Erreurs	409 (pas d'épisode actif)

5.4 `GET /snapshot` — Dernière observation cachée

Purpose	Retourne la dernière `ObservationResponse` en cache. Utile pour les spectateurs rejoignant en cours de session.
Response	JSON brut de la dernière observation
Erreurs	404 (aucune observation encore émise)

5.5 `GET /telemetry` — Journal de vol

Response	`TelemetryResponse` :
	`total_steps`, `total_distance`, `clips_count`, `altitude_range`
	`records[] : {step, x, y, z, dx, dy, dz, ground_footprint, was_clipped, reason}`

5.6 GET /config — Configuration du simulateur

Response	ConfigResponse : data_dir, roi, n_tiles, pixel_size_m, tile_ground_size map_bounds : {x_min, y_min, x_max, y_max, width, height} drone : {z_min, z_max, default_altitude, max_step_distance} camera : {fov_deg, image_size} max_steps, is_running, scenario_id

5.7 GET /scenarios — Liste des scénarios

Response	ScenariosListResponse : scenarios[] : {scenario_id, name, description, max_steps, target_radius, environment[], difficulty}

5.8 GET /overview — Image vue d'ensemble

Query params	?size=1024 (int) — taille de l'image en pixels
Response	JPEG binaire avec headers personnalisés : X-Bounds-Xmin, X-Bounds-Ymin, X-Bounds-Xmax, X-Bounds-Ymax
Note	Image cachée après la première génération

5.9 GET /events — Flux SSE

Response	text/event-stream
Événements	event: observation + data: <ObservationResponse JSON>
Keep-alive	Commentaire :keep-alive toutes les 30 s
Init	Commentaire :connected immédiat (flush des en-têtes)
Queue	asyncio.Queue(maxsize=64) par abonné. Si pleine, l'événement le plus ancien est supprimé

5.10 Format ObservationResponse

```
{
  "step": 42,
  "done": false,
  "drone_state": {
    "x": 923456.2, "y": 6345123.8, "z": 100.0,
    "heading": 0.0, "step_count": 42,
    "total_distance": 523.4
  },
  "ground_footprint": 200.0,
  "ground_resolution": 0.4,
  "image_base64": "<PNG en base64>",
  "image_width": 500,
  "image_height": 500,
  "result": null,
  "metadata": {
    "roi": "UU-S2-1",
    "primary_modality": "AERIAL_RGBI",
    "distance_to_target": "42.5"
```

```
  },
  "images": {
    "AERIAL_RGBI": "<base64 PNG>",
    "DEM_ELEV": "<base64 PNG>"
  }
}
```

Listing 1 – Structure complète d'une ObservationResponse

Lorsque `done=true`, le champ `result` contient :

```
{
  "success": true,
  "reason": "Agent declared FOUND within 42.5 m of target",
  "steps_taken": 42,
  "distance_travelled": 523.4
}
```

6 API de l'orchestrateur web

Tous les endpoints sont préfixés par `/api/` sauf la redirection racine et les fichiers statiques.

6.1 `GET /` — Redirection vers la SPA

Redirection 302 vers `/static/index.html`.

6.2 `GET /api/status` — Health check

```
{ "status": "ok", "active_sessions": 3, "scenarios_dir": "scenarios/" }
```

6.3 `GET /api/scenarios` — Liste des scénarios

Retourne tous les scénarios avec détails complets (dataset, start, target, prompt, environment, difficulty).

6.4 `GET /api/scenarios/{id}` — Détail d'un scénario

Retourne un scénario avec ses templates de prompt pour les agents VLM.

6.5 `GET /api/scenarios/{id}/overview` — Image overview

Retourne l'image JPEG de vue d'ensemble pré-générée avec headers `X-Bounds-*`.

6.6 POST /api/sessions — Créer une session

Request body	`scenario_id` : string (requis) `mode` : `"human"` (défaut) | `"ai"` `player_name` : string|null `model_info` : object|null — métadonnées du modèle AI
Auth	Mode AI : header `Authorization: Bearer <FLAIRSIM_API_KEY>` requis Mode human : aucune authentification
Response	`{session_id, scenario_id, mode, port, status: "ready", created_at, base_url}`
Erreurs	400 (champs manquants), 401 (clé API invalide), 404 (scénario introuvable), 500 (échec spawn)
Effets	Spawn un sous-processus `python -m flairsim.server`, health check GET /config pendant 30 s max

6.7 GET /api/sessions — Lister les sessions

Retourne `{"sessions": [...]}`.

6.8 GET /api/sessions/{id} — État d'une session

Vérifie aussi si le sous-processus est toujours vivant.

6.9 DELETE /api/sessions/{id} — Détruire une session

Tue le sous-processus, libère le port.

6.10 ANY /api/sessions/{id}/sim/{path} — Proxy

Méthodes	GET, POST, PUT, DELETE, PATCH
Purpose	Relaye toute requête vers le sous-processus simulateur
SSE	Quand `path == "events"` et GET : proxy SSE streaming via `httpx.AsyncClient.stream()`
Activité	Appelle `touch_session()` à chaque requête
Erreurs	404 (session introuvable), 502 (simulateur en erreur), 503 (encore en démarrage), 504 (timeout)

6.11 GET /api/leaderboard — Classement

Query params	`?scenario_id=...`, `?mode=human|ai`, `?limit=50`
Tri	`ORDER BY success DESC, steps_taken ASC, distance_travelled ASC`
Response	`{"runs": [...]}`

6.12 POST /api/leaderboard — Soumettre un run

Request body	`scenario_id` (requis), `mode` (requis)
	`player_name`, `model_name`, `success`, `reason`
	`steps_taken`, `distance_travelled`, `duration_s`
	`trajectory`, `steps_detail`, `model_info`, `metrics`, `score_final`

Response	`{"run_id": "<UUID>"}`

6.13 POST /api/leaderboard/submit — Alias

Alias pour POST /api/leaderboard. Endpoint préféré pour les notebooks.

6.14 GET /api/leaderboard/{id} — Détail d'un run

Retourne l'objet complet avec colonnes JSON désérialisées.

7 Leaderboard — Schéma SQLite

```sql
CREATE TABLE IF NOT EXISTS runs (
  id                  TEXT PRIMARY KEY,  -- UUID
  session_id          TEXT,
  scenario_id         TEXT NOT NULL,
  player_name         TEXT,
  model_name          TEXT,
  mode                TEXT NOT NULL CHECK (mode IN ('human','ai')),
  success             INTEGER NOT NULL DEFAULT 0,
  reason              TEXT,
  steps_taken         INTEGER,
  distance_travelled  REAL,
  duration_s          REAL,
  trajectory          TEXT,      -- JSON serialized
  steps_detail        TEXT,      -- JSON serialized
  model_info          TEXT,      -- JSON serialized
  metrics             TEXT,      -- JSON serialized
  score_final         REAL,
  created_at          TEXT NOT NULL  -- ISO 8601
);
CREATE INDEX IF NOT EXISTS idx_runs_scenario
  ON runs (scenario_id);
```

Listing 2 – Schéma de la table runs

Configuration : journal_mode=WAL, check_same_thread=False. Migration automatique pour ajouter les colonnes manquantes.

8 Format des scénarios YAML

```yaml
scenario_id: find_target_D006
name: Find target in Isere
description: Navigate to locate a ground target

dataset:
  data_dir: D006-2020_AERIAL_RGBI
  domain: D006-2020
  roi: UU-S2-1
  modalities:
    - AERIAL_RGBI
  source: auto        # local | huggingface | auto

start:
  x: 1018422.0
  y: 6278750.0
  z: 150.0

target:
  x: 1018300.0
```

```yaml
  y: 6278200.0
  radius: 50.0          # metres

max_steps: 200
environment: [rural]
difficulty: 2            # 1-3

prompt:
  system: |
    You are a drone navigation agent...
  user_template: |
    Position: ({x}, {y}, {z}). Step {step}/{max_steps}.
```

Listing 3 – Structure d'un scénario YAML

Placeholders disponibles dans `user_template` : `{x}`, `{y}`, `{z}`, `{step}`, `{max_steps}`, `{steps_remaining}`, `{distance}`.

9 Diagrammes de séquence

9.1 Création d'une session humaine (navigateur)

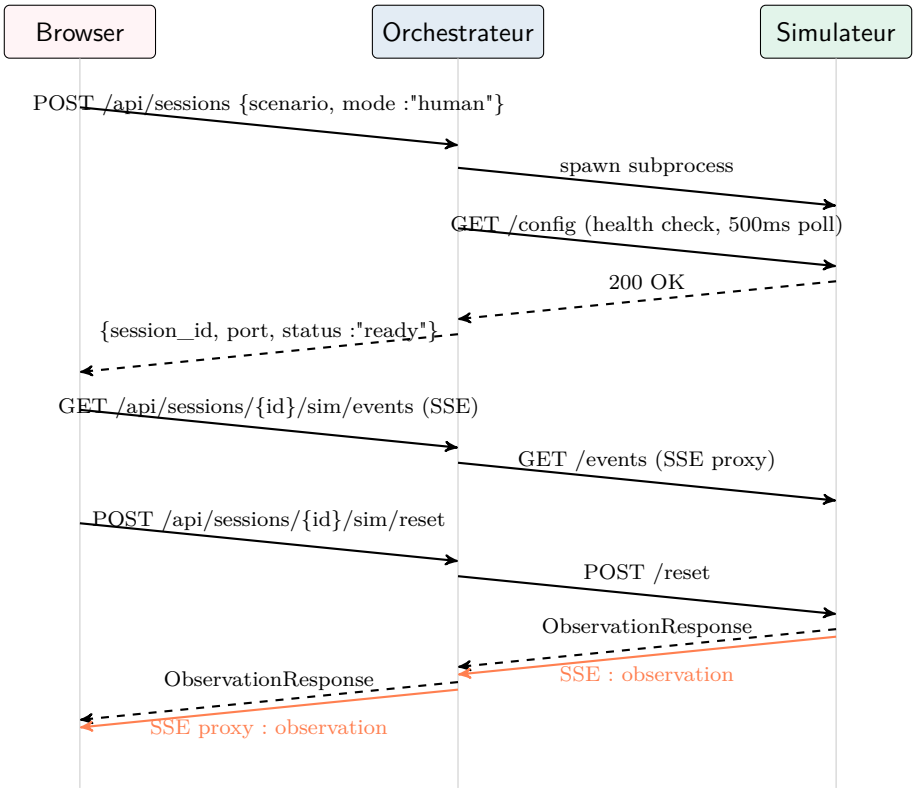

Figure 5 – Séquence : création d'une session humaine

9.2 Mouvement fluide (smooth step) via SSE

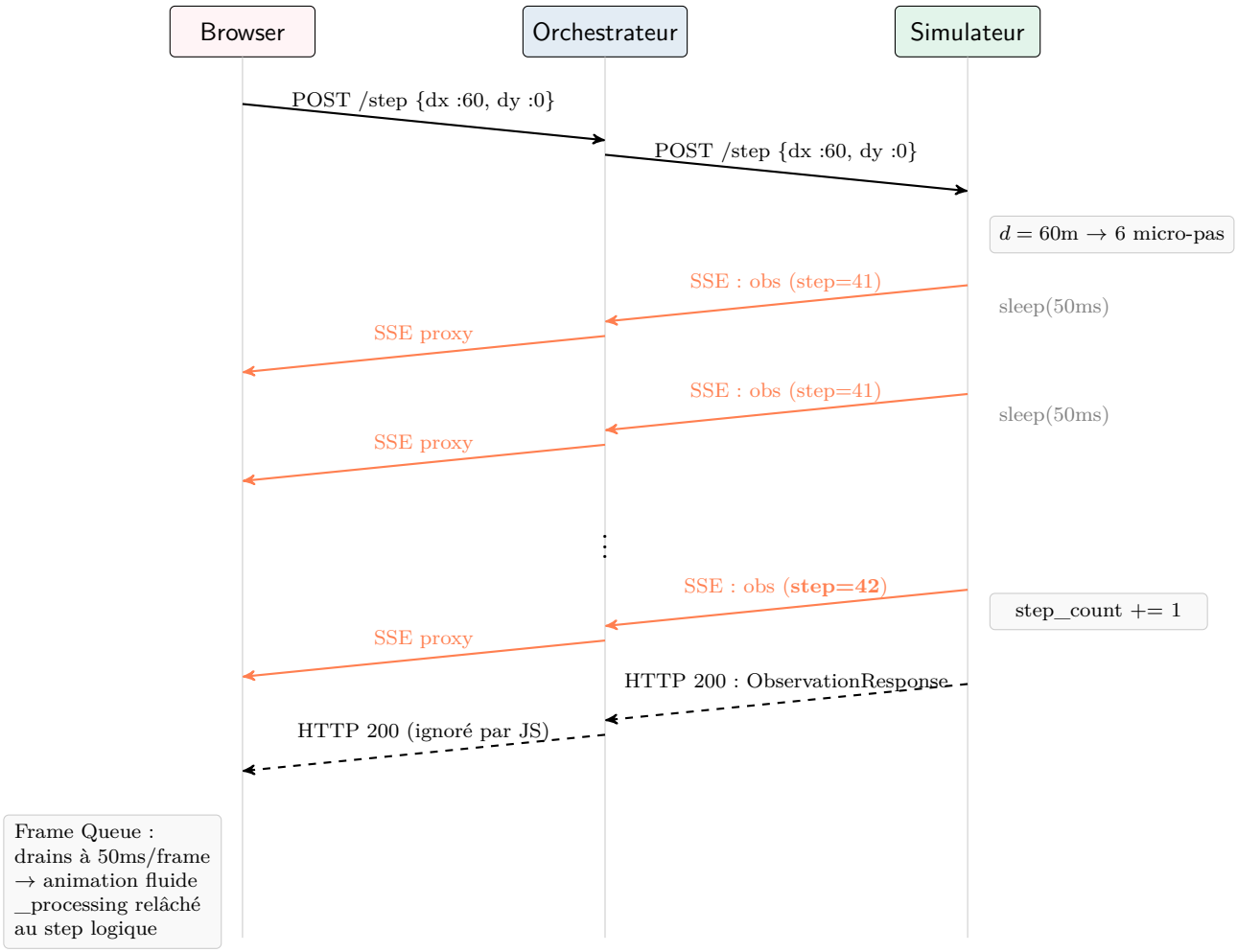

Figure 6 – Séquence : mouvement fluide avec micro-pas SSE

9.3 Session AI depuis un notebook

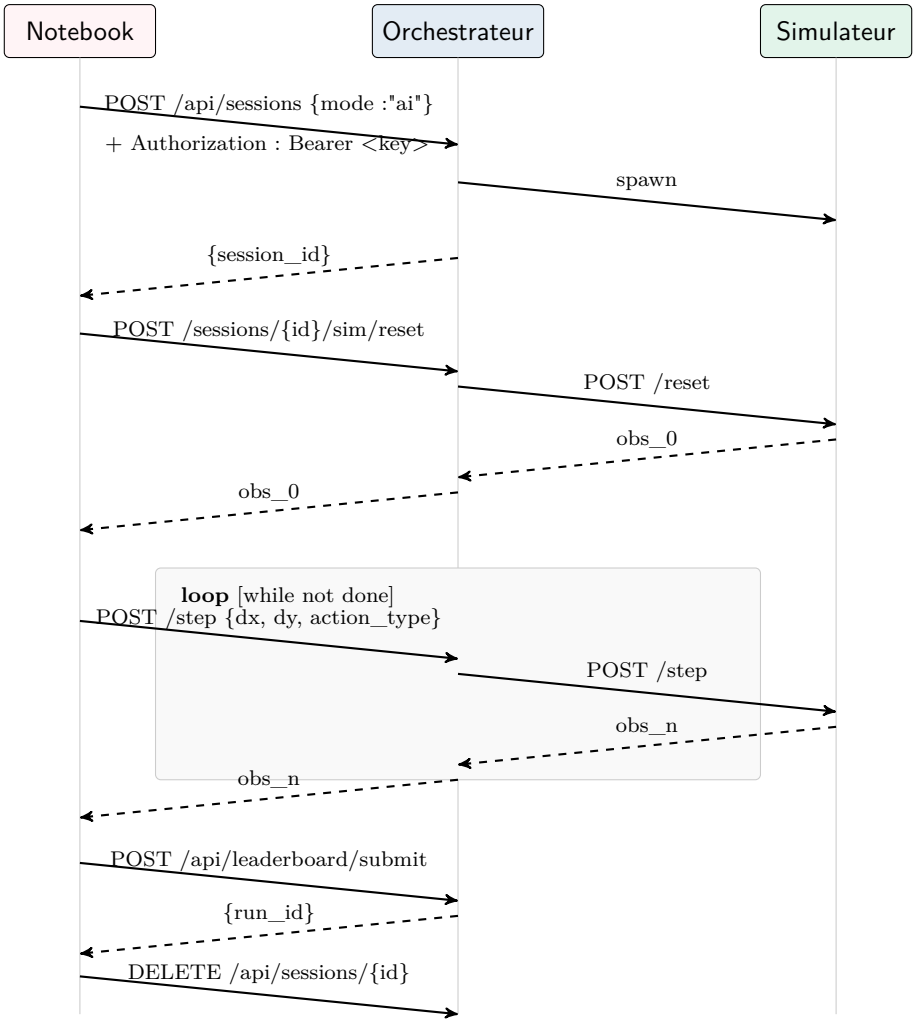

Figure 7 – Séquence : boucle complète d'un agent AI depuis un notebook

9.4 Spectateur rejoignant une session en cours

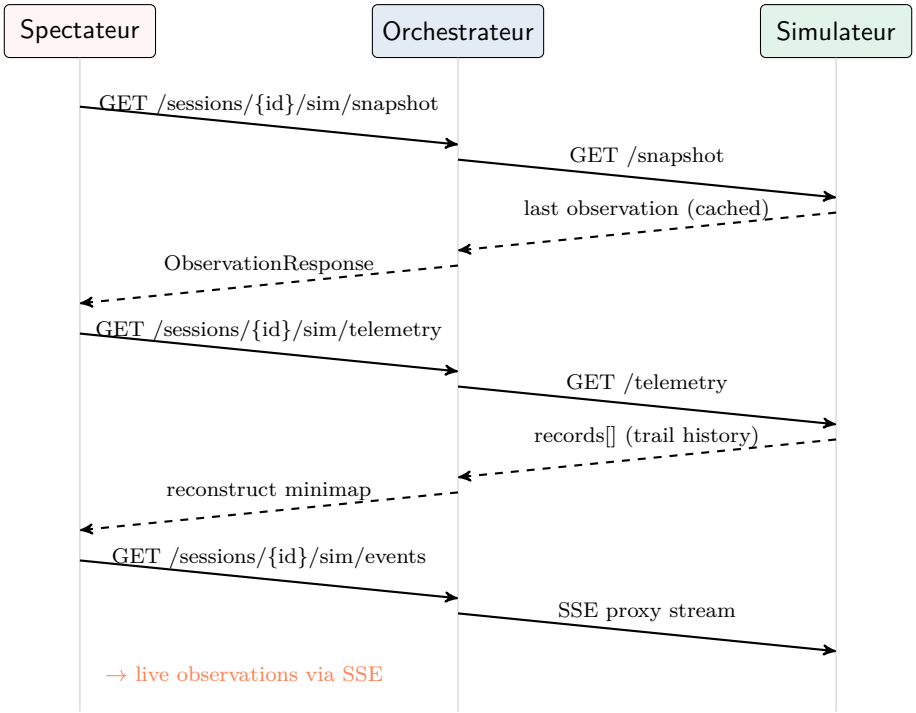

Figure 8 – Séquence : spectateur rejoignant en cours de session

9.5 Flux de données complet d'un pas de simulation

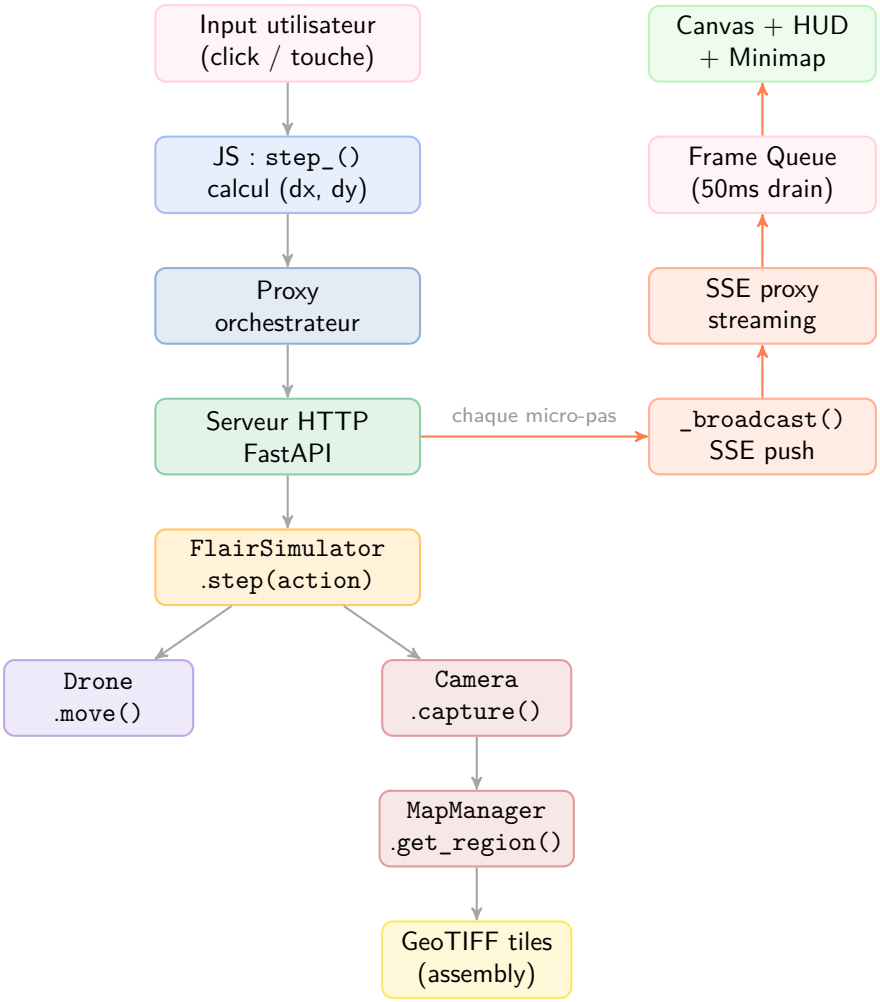

Figure 9 – Flux de données complet : de l'input utilisateur au rendu

10 Authentification API

1. **Source de la clé** : variable d'environnement `FLAIRSIM_API_KEY` ou fichier `.env` à la racine.
2. **Portée** : seul `POST /api/sessions` en mode `"ai"` exige la clé.
3. **Header** : `Authorization: Bearer <token>`.
4. **Mode human** : aucune authentification requise.
5. **Sessions existantes** : les requêtes proxy ne sont pas authentifiées (la session est déjà créée).

Si aucune clé n'est configurée : le serveur retourne 503 pour les sessions AI ("AI API access not configured").

11 Données FLAIR-HUB

11.1 Modalités supportées

Modalité	Suffixe	Résolution	Bandes	Type
AERIAL_RGBI	`AERIAL_RGBI`	0.2 m/px	4	uint8
AERIAL_RLT_PAN	`AERIAL-RLT_PAN`	0.4 m/px	1	uint8
DEM_ELEV	`DEM_ELEV`	0.2 m/px	2	float32
SPOT_RGBI	`SPOT_RGBI`	1.6 m/px	4	uint16
SENTINEL1_ASC	`SENTINEL1-ASC_TS`	10.24 m/px	2	float32
SENTINEL1_DESC	`SENTINEL1-DESC_TS`	10.24 m/px	2	float32
SENTINEL2_TS	`SENTINEL2_TS`	10.24 m/px	10	uint16
LABEL_COSIA	`AERIAL_LABEL-COSIA`	0.2 m/px	1	uint8
LABEL_LPIS	`ALL_LABEL-LPIS`	0.2 m/px	3	uint8
SENTINEL2_MSK	`SENTINEL2_MSK-SC`	10.24 m/px	2	uint16

Table 7 – Modalités FLAIR-HUB (tous les patches couvrent 102.4 m × 102.4 m)

11.2 Téléchargement automatique

Si `-domain D006-2020` est spécifié sans `-data-dir` :

1. `HubDownloader` télécharge depuis `IGNF/FLAIR-HUB` sur HuggingFace.
2. Les ZIP sont extraits dans un répertoire temporaire.
3. Nettoyage automatique à l'arrêt (SIGINT/SIGTERM).

12 Grille overlay pour VLMs

La classe `GridOverlay` divise l'image en une grille $N \times N$ avec des labels alphanumériques (A1, A2, B1, ...), inspiré des approches **Set-of-Mark** [3] et **Scaffold** [4].

Points d'intégration :
- API serveur : `?grid=N` sur `/reset` et `/step`
- Viewer : `-grid N` + touche G
- Programmatique : `GridOverlay(n=4).draw(image)`

13 Installation et utilisation

13.1 Installation

```
cd simulateur
uv sync --all-extras    # core + server + viewer + dev
```

13.2 Lancer le simulateur (mode standalone)

```
# Donnees locales
uv run python -m flairsim.server --data-dir path/to/D004_AERIAL_RGBI

# Auto-download depuis HuggingFace
uv run python -m flairsim.server --domain D006-2020

# Avec scenario
uv run python -m flairsim.server \
    --data-root path/to/data --scenarios-dir scenarios/ \
    --scenario find_target_D006
```

13.3 Lancer la plateforme web

```
uv run python -m flairsim.web \
    --scenarios-dir scenarios/ \
    --data-root path/to/data
```

Ouvrir `http://127.0.0.1:8080` dans le navigateur.

13.4 Viewer Pygame

```
# Mode local
uv run python -m flairsim.viewer --data-dir path/to/data

# Mode observe (SSE)
uv run python -m flairsim.viewer --mode observe

# Mode fly (pilotage distant)
uv run python -m flairsim.viewer --mode fly
```

13.5 Contrôles clavier

Touche	Action	Touche	Action
Z / W / ↑	Nord	E	Monter
S / ↓	Sud	A	Descendre
D / →	Est	+/−	Taille du pas
Q / ←	Ouest	Espace	FOUND
Tab	Modalité	R	Reset
H	HUD	M	Minimap
G	Grille	Échap	Quitter

Table 8 – Contrôles clavier (web + viewer Pygame)

14 Tests

La suite comprend **554 tests** couvrant tous les modules :

```
uv run pytest            # 554 tests, ~9s
uv run pytest -v         # verbeux
uv run pytest -x         # arret au premier echec
```

Fichier	Tests	Portée
test_action.py	13	Action, ActionType
test_camera.py	15	CameraModel
test_drone.py	39	Drone, DroneConfig
test_map_manager.py	44	MapManager (GeoTIFFs synthétiques)
test_modality.py	11	Modality, ModalitySpec
test_multimodality.py	12	Multi-modality (intégration)
test_observation.py	14	Observation, EpisodeResult
test_scenario.py	49	Scenario, ScenarioLoader
test_server.py	48	Endpoints HTTP + SSE + Snapshot
test_simulator.py	47	FlairSimulator (intégration)
test_telemetry.py	16	TelemetryRecord, FlightLog
test_tile_loader.py	16	TileInfo, TileData
test_downloader.py	22	HubDownloader (mock HF)
test_grid.py	59	GridOverlay
test_sessions.py	52	SessionManager (mock subprocess)
test_leaderboard.py	30	Leaderboard SQLite
test_web_routes.py	67	Routes orchestrateur
Total	**554**	

Table 9 – Répartition des tests par fichier

15 Structure du projet

```
simulateur/
  flairsim/                   # Package principal
    __init__.py               # API publique (26 exports)
    data/                     # Telechargement HuggingFace
      downloader.py           # HubDownloader
    map/                      # Tuiles GeoTIFF, requetes spatiales
      modality.py             # Enum Modality (10 modalites)
      tile_loader.py          # TileInfo, TileData, read_tile()
      map_manager.py          # MapManager, MapBounds
    drone/                    # Etat du drone, camera, telemetrie
      drone.py                # Drone, DroneConfig, DroneState
      camera.py               # CameraModel, CameraConfig
      telemetry.py            # FlightLog, TelemetryRecord
    core/                     # Moteur de simulation
      action.py               # Action, ActionType (MOVE/FOUND/STOP)
      observation.py          # Observation, EpisodeResult
      simulator.py            # FlairSimulator, SimulatorConfig
      scenario.py             # Scenario, ScenarioLoader, ScenarioPrompt
      grid.py                 # GridOverlay (SoM/Scaffold)
    server/                   # API HTTP REST + SSE (simulateur)
      app.py                  # FastAPI factory, 9 endpoints, SSE
      cli.py                  # CLI parsing, auto-download
    web/                      # Plateforme web (orchestrateur)
      app.py                  # FastAPI, routes, proxy, middleware
      sessions.py             # SessionManager (subprocess, ports)
      leaderboard.py          # SQLite CRUD, ranking, migration
      cli.py                  # CLI parsing
      static/                 # SPA vanilla JS
        index.html            # 5 vues + modal resultats
        style.css             # Theme sombre, responsive (1188 lignes)
        app.js                # Router, GameEngine, Canvas, Minimap (1648 lignes)
    viewer/                   # Visualisation Pygame
      viewer.py               # FlairViewer (3 modes)
      remote.py               # ViewerObservation adapter
      minimap.py              # Minimap overlay
      hud.py                  # HUD overlay
  scenarios/                  # Fichiers YAML de scenarios
  tests/                      # 554 tests (17 fichiers)
  docs/                       # Documentation technique
  pyproject.toml              # Config package (hatchling)
```

16 Références

Références

[1] Music, A. et al. (2025). *FLAIR-HUB : A large-scale multimodal dataset for land-cover mapping.* arXiv :2506.07080.

[2] Majumdar, A. et al. (2025). *FlySearch : A benchmark for evaluating VLM exploration capabilities with a UAV.* NeurIPS 2025. arXiv :2506.02896.

[3] Yang, J. et al. (2023). *Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V.* arXiv :2310.11441.

[4] Lei, L. et al. (2024). *Scaffolding coordinates to promote vision-language coordination in large multi-modal models.* arXiv :2402.12058.